AI AGENT

ShopWave

Autonomous Support Resolution Agent

ReAct	5	10×
Engine	Core Constraints	Max Iterations/Ticket

Hackathon 2026 • E-Commerce AI Track

Project Overview

🤖 What It Does

Fully autonomous Tier-2 customer support agent that reads tickets, reasons using a policy knowledge base, executes tools, and resolves issues — without human involvement.

🧠 How It Thinks

Implements the ReAct pattern: alternating Thought → Action → Observation loops with explicit state tracking and policy citations in every reasoning step.

⚙️ Tech Stack

Python async (asyncio), Gemini 2.5 Flash for LLM reasoning, Pydantic v2 for schema validation, Tenacity for retry logic, and CircuitBreaker for fault tolerance.

📋 Output & Auditability

Every resolved or escalated ticket produces a structured audit log entry with reasoning trace, tool call evidence, confidence score, and correlation ID.

System Architecture

Orchestrator

main.py

- TicketIngester reads tickets.json
- asyncio.gather() for concurrency
- Max 5 parallel AgentWorkers
- Throttler for rate limiting

ReAct Engine

src/agent.py

- State machine with 8 states
- Max 10 iterations per ticket
- Tool call guard (min 3 before action)
- Reasoning trace logger

Tool Layer

src/tools.py

- get_customer / get_order / get_product
- search_knowledge_base
- check_refund_eligibility
- issue_refund / send_reply / escalate

All decisions → audit_log.json | Failures → dead_letter.json | Full Pydantic schema validation at I/O boundaries

ReAct Loop: How the Agent Thinks

1	STATE: INGESTING	**Action:**	*get_customer(email)*	→ **Obs:**	tier=vip, id=C001
2	STATE: CUSTOMER_VERIFIED	**Action:**	*get_order(order_id)*	→ **Obs:**	status=delivered, amount=129.99
3	STATE: ORDER_VERIFIED	**Action:**	*get_product(product_id)*	→ **Obs:**	category=electronics, warranty=12mo
4	STATE: POLICY_CHECKED	**Action:**	*check_refund_eligibility()*	→ **Obs:**	eligible=true, threshold OK
5	STATE: ACTION_TAKEN	**Action:**	*issue_refund() → send_reply()*	→ **Obs:**	resolved / refund_issued

Every Thought includes: [STATE] + [POLICY citation] + explicit reasoning — mandatory before each tool call

5 Non-Negotiable Constraints

01

Deep Tool Chaining

Minimum 3 verification tools must be called before any terminal action (refund/escalate). Enforced by a system guard in agent.py.

02

True Concurrency + Failure Isolation

asyncio.gather(..., return_exceptions=True) dispatches up to 5 tickets in parallel. Each AgentWorker is fully isolated — one crash doesn't affect others.

03

Tiered Resilience

3-tier safety net: Tenacity retry/backoff → Circuit Breaker (prevents cascade failures) → Dead-letter queue for unrecoverable tickets.

04

Full Explainability

Every decision step logs a [STATE] + [POLICY] tagged Thought. Reasoning traces are captured in audit_log.json for post-hoc review.

05

Schema Validation Before Action

All tool inputs and outputs are Pydantic v2 models. execute_tool_safely() normalizes and validates every response before the agent can act on it.

Tool Ecosystem

👤 Lookup

get_customer

Fetches customer tier (Standard/Premium/VIP) and account notes. First mandatory step.

📦 Lookup

get_order / get_product

Retrieves order status, delivery date, product category, and warranty info.

📖 Policy

search_knowledge_base

Keyword search over the KB markdown for return, refund, and escalation policies.

✅ Decision

check_refund_eligibility

Circuit-breaker protected call. Returns eligibility, reason, and escalation flag.

💳 Terminal Action

issue_refund

Irreversible. Guarded by min-tool-call check + schema validation before execution.

📨 Terminal Action

escalate / send_reply

Routes ticket to human agent with priority or sends final customer reply.

Resilience & Fault Tolerance

Tier 1 — Retry + Backoff

Tenacity wrappers with exponential backoff + jitter on transient errors. Retry count tracked per ticket in audit log.

Tier 2 — Circuit Breaker

check_refund_eligibility uses circuitbreaker library. When open, tickets are fast-failed and rerouted without hammering a degraded service.

Tier 3 — Dead-Letter Queue

Policy violations and unrecoverable errors are written to dead_letter.json. Never silently dropped — always capturable for manual review.

Live Run: 20 tickets • 1 ticket triggered retries (3 total retries) • 0 circuit breaker trips • 0 dead-lettered

Audit & Explainability

ticket_id / correlation_id

Unique trace token linking all logs for one ticket

reasoning_trace

Full [STATE] + [POLICY] Thought log — explainability ledger

tools_called

Every tool call with args, success flag, and retry count

final_action + status

Terminal outcome: resolved, escalated, needs_clarification

confidence_score

Agent's self-assessed certainty (0–1) used for routing decisions

processing_time_ms

End-to-end latency per ticket

Sample Run Summary

Model	**Gemini 2.5 Flash**
Total Tickets	**20**
Resolved	**0**
Escalated	**20**
Avg Processing	**~51s/ticket**
Avg Tool Calls	**4.6/ticket**
Retried Tickets	**1 (3 retries)**
CB Trips	**0**
Eval Score	**54% pass rate**

Limitations & Roadmap

Current Limitations

- Keyword-only KB search (no vector/semantic retrieval)
- Multi-order disambiguation is model-led, not tool-enforced
- No external fraud service — relies on prompt-guided reasoning
- Audit files rewritten on append (not optimized for scale)
- Heuristic eval scorer (free-text label bucketing)

Roadmap

- **Vector KB search**
  Embeddings + semantic similarity for policy lookup
- **External fraud API**
  Integrate dedicated fraud scoring service
- **Streaming audit writes**
  Append-only log file with proper streaming I/O
- **Multi-model support**
  Swap LLM backend (Claude, GPT-4) via adapter layer
- **Human-in-the-loop UI**
  Dashboard for reviewing dead-lettered tickets

ShopWave

Autonomous. Auditable. Resilient.

ReAct Engine | Circuit Breaker | Pydantic Schemas | Audit Logs | Tier Policies

Thank You

Questions? Let's dig into the code!